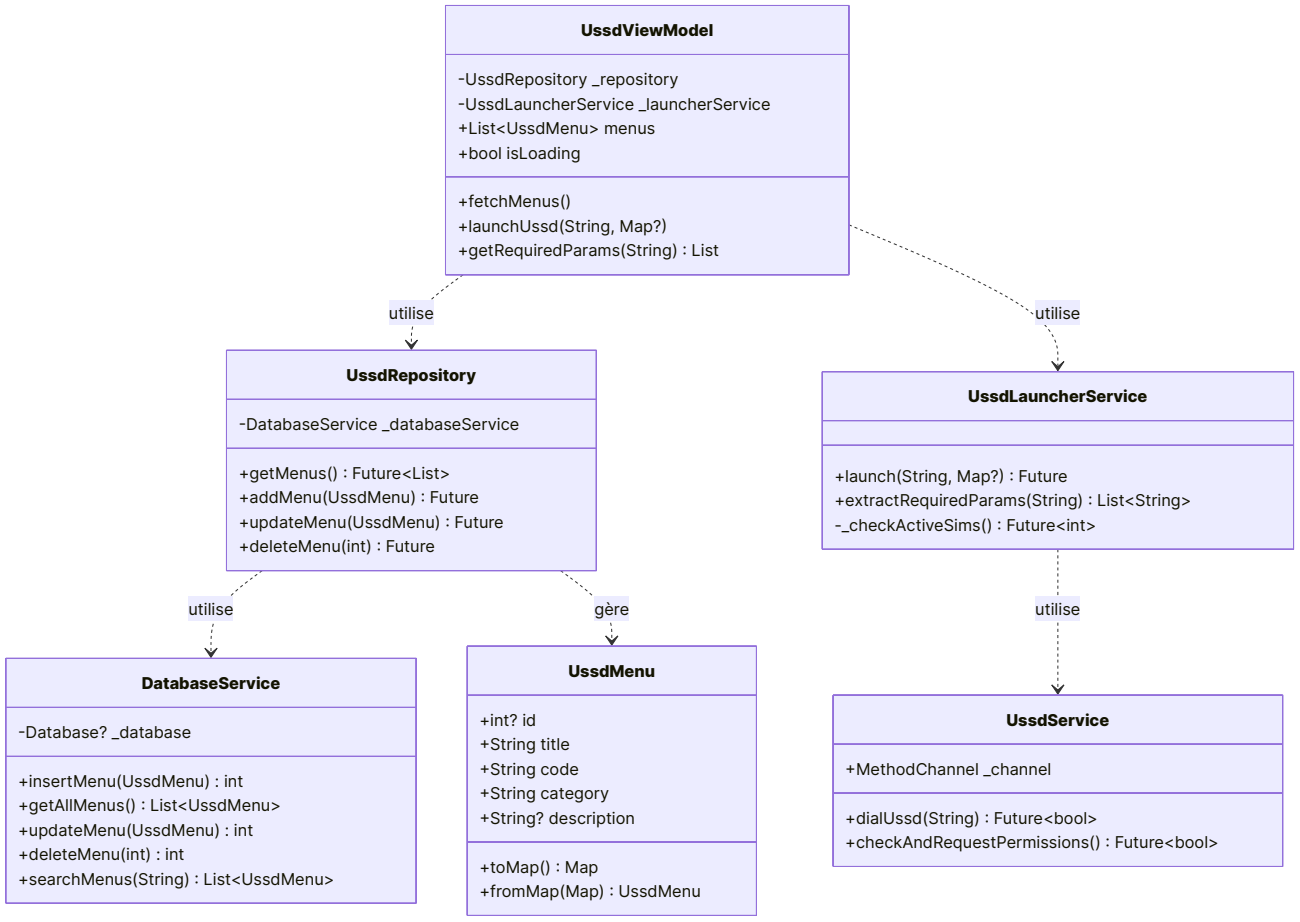
# Rapport de TP : Conception d'une Application de Gestion de Menus USSD

## 1. Diagramme de Classe (UML)

Le diagramme suivant illustre l'architecture **MVVM** mise en place. Il montre la séparation entre les données ( `UssdMenu` ), l'accès aux données ( `DatabaseService` , `UssdRepository` ), la logique métier ( `UssdLauncherService` , `UssdService` ) et la gestion de l'état ( `UssdViewModel` ).



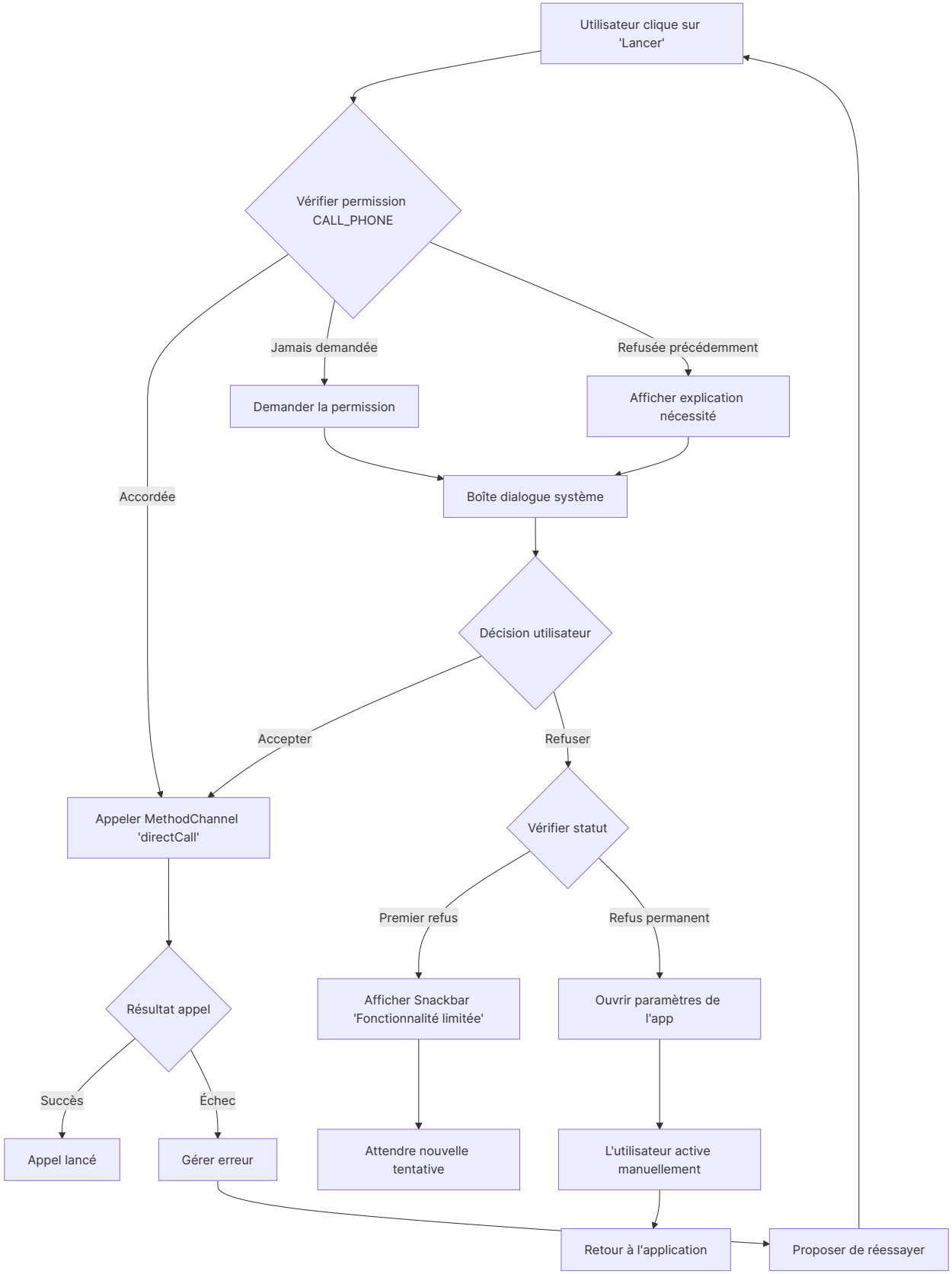

## 2. Architecture des Dossiers

Le projet est structuré selon les principes de la **Clean Architecture** simplifiée pour Flutter, favorisant la maintenabilité et la testabilité.

```
lib/
├── main.dart                   # Point d'entrée, configuration du Provider
├── models/
│   └── ussd_menu.dart          # Modèle de données (Entité)
├── repositories/
│   └── ussd_repository.dart    # Abstraction de la source de données
├── screens/
│   ├── dashboard_screen.dart   # Vue principale (Liste & Recherche)
│   └── edit_menu_screen.dart   # Vue formulaire (Ajout & Modification)
├── services/
│   ├── database_service.dart   # Gestion bas niveau SQLite
│   ├── ussd_launcher_service.dart # Logique métier (Multi-SIM, Params)
│   └── ussd_service.dart       # Communication avec le natif (MethodChannel)
├── viewmodels/
│   └── ussd_viewmodel.dart     # Gestion d'état (Lien Vue <-> Logique)
└── widgets/                    # Composants réutilisables (si nécessaire)
```

## 3. Logique des Permissions

L'application interagit avec le composeur téléphonique natif, ce qui est une action sensible. Voici le flux de demande d'autorisation implémenté dans `UssdService` :




**Permissions déclarées dans AndroidManifest.xml :**

- `android.permission.CALL_PHONE` : Pour initier l'appel USSD.
- `android.permission.READ_PHONE_STATE` : Nécessaire pour la gestion Multi-SIM (lecture des abonnements).

# 4. Justification des Choix Techniques

## 4.1. Choix de la Base de Données : SQLite (via sqflite)

Nous avons choisi **SQLite** plutôt que *SharedPreferences* ou *Hive* pour les raisons suivantes :

1. **Structure Relationnelle** : Les données "Menu USSD" sont fortement structurées (ID, Titre, Code, Catégorie). SQLite force cette structure via un schéma de table, garantissant l'intégrité des données.

2. **Requêtes Complexes** : L'application nécessite des fonctionnalités de recherche (filtrer par titre). Le langage SQL ( `WHERE title LIKE %...%` ) est natif et performant pour ce type d'opération, contrairement à SharedPreferences qui nécessiterait de charger toutes les données en mémoire pour les filtrer.
3. **Scalabilité** : Si l'application évolue pour gérer des historiques d'appels ou des relations entre catégories, SQLite est le seul choix viable parmi les solutions locales légères.

## 4.2. Architecture d'État : Provider & MVVM

Nous avons opté pour le pattern **MVVM** couplé au package **Provider** :

1. **Séparation des Responsabilités** :
   - La **Vue** (Screens) ne fait qu'afficher l'état et capturer les événements.
   - Le **ViewModel** contient toute la logique de présentation et l'état mutable ( `isLoading` , `menus` , `errorMessage` ).
   - Le **Modèle/Repository** gère les données pures.
2. **Pourquoi Provider ?** :
   - C'est la solution recommandée par l'équipe Flutter pour la gestion d'état intermédiaire.
   - Il est plus léger et moins "verbeux" que **BLoC** pour une application CRUD simple.
   - Il offre une injection de dépendance simple pour rendre le `UssdViewModel` accessible partout dans l'arbre des widgets.
3. **Gestion des Erreurs** : Le ViewModel centralise la gestion des exceptions (ex: échec de l'appel USSD), permettant à l'UI de réagir simplement en observant les changements de variables.